

Prompt Engineering

Mehmet Ali Akyol, PhD

Ankara Medipol University

November 27, 2025

Week 6: Iterative Prompt Refinement

Where We Are

- **Week 4:** We built a library of patterns (CO-STAR, RACI) to structure our requests.
- **Week 5:** We mastered style, tone, and formatting to control *how* the model speaks.
- **This Week:** We shift from “writing” to “engineering.”
- **Goal:** Turn prompt creation into a systematic, data-driven optimization process.

Weekly Learning Objectives

- Diagnose prompt failures using a structured error taxonomy.
- Design “Critique Prompts” that act as automated editors.
- Implement “LLM-as-a-Judge” workflows to scale evaluation.
- Build multi-turn refinement loops (Draft → Critique → Refine).
- Create a “Prompt Runbook” to document version history and performance.

The Prompt Engineering Lifecycle

Draft (Weeks 1–5) → **Evaluate** (Week 6) → **Refine** (Week 6) → **Deploy**

- **Draft:** Apply frameworks (CO-STAR) and style guides.
- **Evaluate:** Check against requirements (Accuracy, Tone, Format).
- **Refine:** Adjust instructions, add constraints, or provide few-shot examples.
- **Deploy:** Version control and monitor for drift.

Error Taxonomy: Naming the Problem

1. Reasoning Failures

- Logic gaps, math errors, hallucinated facts.
- *Fix*: Chain-of-Thought, external tools.

2. Constraint Violations

- Ignoring word counts, wrong format (text vs JSON).
- *Fix*: Negative constraints, structural contracts (Week 5).

3. Tone/Style Drift

- Too casual, too robotic, wrong audience level.
- *Fix*: Voice fingerprinting, reference samples.

4. Phrasing Sensitivity

- Model confused by ambiguous terms.
- *Fix*: Clearer definitions, synonyms.

Root Cause Analysis: The “Why”

- **Ambiguity:** Did you ask for a “short summary” (vague) or “3 bullet points” (specific)?
- **Context Overload:** Is the model distracted by irrelevant information in the prompt?
- **Conflicting Directives:** “Be detailed” AND “Keep it under 50 words.”
- **Model Limitations:** Asking a small model to do complex reasoning without steps.

The Critique Loop

Concept

Instead of guessing how to fix a prompt, ask the model to critique its own output against a rubric.

- **Step 1: Generate.** Run your draft prompt.
- **Step 2: Critique.** Feed the output back to the model with a “Critique Prompt.”
- **Step 3: Revise.** Ask the model to rewrite the output based on the critique.
- **Why?** It separates “creation” (high creativity) from “editing” (high logic).

Deep Dive: The Critique Prompt Structure

Template

“You are an expert editor. Review the text above against the following criteria:

- ① **Tone:** Is it professional and empathetic? (Week 5)
- ② **Structure:** Does it follow the CO-STAR framework? (Week 4)
- ③ **Accuracy:** Are all claims supported by the provided context?

Provide a score (1–5) for each and list specific improvements.”

Key Insight: Use your frameworks (CO-STAR, RACI) as the grading rubric.

Iterative Workflow Patterns

- **Single-loop (Self-Correction):** Draft → Critique → Revise. Good for simple tasks.
- **Double-loop (Prompt Refinement):** Use the critique to rewrite the *prompt itself*, not just the output.
- **Parallel Exploration (Best-of-N):** Generate 5 variants, critique all, pick the winner.
- **Human-in-the-loop:** Model critiques → Human approves plan → Model executes.

What is “Vibe Coding”?

Definition

Writing code by describing the desired behavior (the “vibe”) to an LLM, then iterating based on the output’s look and feel rather than inspecting the source code.

- **Natural Language as Syntax:** You don’t write Python; you write English. The LLM translates.
- **Human Role Shift:** You move from **Implementer** to **Curator**.
- **The Loop:** Prompt → Run → “Vibe Check” (Does it work?) → Iterate.
- **Key Characteristic:** Speed and intuition over precision and structure.

The “Vibe Coding” Trap

- **The Pro:** Incredible velocity for prototypes and simple tools.
- **The Con: Fragility.** If you don't understand the code, you can't fix edge cases.
- **The Engineering Gap:**
 - “It works on my machine” → “It vibes on my screen.”
 - **Reality Check:** Production systems need **Evals**, not just vibes.

Instrumentation: Measuring Success

- **Version Control:** Treat prompts like code (git). Track changes.
- **Golden Dataset:** Maintain 10–20 “perfect” examples to test against.
- **Metric Tracking:**
 - *Pass Rate:* % of outputs requiring no manual edit.
 - *Latency:* Time to first token (operational metric).
 - *Token Count:* Cost efficiency.

Concept

Using a strong model (e.g., GPT-4) to grade the outputs of a faster/cheaper model (e.g., GPT-3.5 or Llama).

- **Scalability:** Evaluate thousands of outputs in minutes.
- **Consistency:** Removes human fatigue and variability.
- **Cost:** Cheaper than human labeling for initial passes.

Deep Dive: The Judge Prompt

Template

“You are an impartial judge. Evaluate the Assistant’s response to the User’s question.

User Question: [Insert Input] **Assistant Response:** [Insert Output] **Ground Truth:** [Insert Correct Answer]

Score the response from 1 to 5 based on: 1. Correctness (Is it factually true?) 2. Completeness (Did it answer all parts?)

Output format: JSON {‘score’: int, ‘reasoning’: string}”

Prompt Version Control (PromptOps)

- **The Problem:** Prompts are code, but often treated like text files.
- **Best Practices:**
 - Store prompts in a repo (Git).
 - Use variables for dynamic parts (e.g., `{{user_input}}`).
 - Log every run: Prompt Version + Input + Output + Model Config.
- **A/B Testing:** Run Version A and Version B on live traffic; compare “Judge” scores.

Key Takeaways

- **Don't just write prompts; engineer systems.**
- **Critique loops** are the secret to high-quality output.
- **Automation** (Judges) allows you to scale from 1 to 1000 prompts.
- **Iterate forever:** The best prompt today is the baseline for tomorrow.

Thank You!

Dr. Mehmet Ali Akyol

mehmetali.akyol@gmail.com

Questions?